

Introducción a la Programación con Python

Semana 3: Funciones

∇Nablabs

Contenido

- 1 ¿Qué es una función?
- 2 Definir y llamar funciones
- 3 Parámetros y argumentos
- 4 Retorno de valores
- 5 Alcance de variables
- 6 Funciones lambda
- 7 Buenas prácticas
- 8 Proyecto: Calculadora modular

¿Qué es una función?

-  Bloque de código con un nombre que realiza una tarea específica
-  Permite reutilizar código sin repetirlo
-  Divide un programa grande en partes más manejables

 `def nombre_funcion(parámetros):`

 Ya has usado funciones: `print()`, `input()`, `len()` son funciones integradas de Python

Definir una función

Sintaxis básica

Se usa la palabra clave `def`, seguida del nombre y paréntesis

```
1 def saludar():  
2     print("¡Hola, mundo!")  
3  
4 # Llamar la función  
5 saludar()
```

➔ Salida

```
¡Hola, mundo!
```

i La función solo se ejecuta cuando es **llamada**, no cuando se define

Llamar una función varias veces

Reutilización

Una función puede llamarse tantas veces como sea necesario

```
1 def separador():  
2     print(" " * 30)  
3  
4 print("Sección 1")  
5 separador()  
6 print("Sección 2")  
7 separador()  
8 print("Sección 3")
```

➔ Salida

Sección 1

Sección 2

Sección 3

¿Qué son los parámetros?

Variables que recibe la función para trabajar con datos externos

```
1 def saludar(nombre):  
2     print(f"¡Hola, {nombre}!")  
3  
4 saludar("Ana")  
5 saludar("Carlos")
```

➔ Salida

```
¡Hola, Ana!  
¡Hola, Carlos!
```

i **Parámetro:** variable en la definición **Argumento:** valor al llamar

Múltiples parámetros

```
1 def presentar(nombre, edad, ciudad):  
2     print(f"Me llamo {nombre}, tengo {edad} años")  
3     print(f"y vivo en {ciudad}.")  
4  
5 presentar("Lucía", 22, "San José")  
6 presentar("Pedro", 35, "Heredia")
```

➔ Salida

Me llamo Lucía, tengo 22 años
y vivo en San José.
Me llamo Pedro, tengo 35 años
y vivo en Heredia.

⚠ El orden de los argumentos debe coincidir con el de los parámetros

Parámetros por defecto ★

Valores predeterminados

Se puede asignar un valor por defecto a un parámetro

```
1 def saludar(nombre, mensaje="¡Bienvenido!"):
2     print(f"Hola, {nombre}. {mensaje}")
3
4 saludar("Ana")
5 saludar("Luis", "¿Cómo estás?")
```

➡ Salida

```
Hola, Ana. ¡Bienvenido!
Hola, Luis. ¿Cómo estás?
```

i Los parámetros con valor por defecto van siempre al **final**

La instrucción return ↩

¿Para qué sirve return?

Hace que la función devuelva un resultado que puede usarse después

```
1 def sumar(a, b):  
2     return a + b  
3  
4 resultado = sumar(3, 5)  
5 print(f"La suma es: {resultado}")  
6 print(f"El doble es: {sumar(10, 10)}")
```

➡ Salida

```
La suma es: 8  
El doble es: 20
```

💡 Una función sin return devuelve None automáticamente

Retornar múltiples valores

```
1 def estadisticas(numeros):
2     total = sum(numeros)
3     promedio = total / len(numeros)
4     maximo = max(numeros)
5     return total, promedio, maximo
6
7 datos = [10, 20, 30, 40]
8 t, p, m = estadisticas(datos)
9 print(f"Total: {t}")
10 print(f"Promedio: {p}")
11 print(f"Máximo: {m}")
```

➔ Salida

Total: 100 Promedio: 25.0 Máximo: 40

Alcance (scope)

Las variables creadas **dentro** de una función son locales y no existen fuera

✖ Error:

```
1 def calcular():  
2     resultado = 42  
3  
4 calcular()  
5 print(resultado)
```

NameError: name 'resultado' is not defined

✔ Correcto:

```
1 def calcular():  
2     resultado = 42  
3     return resultado  
4  
5 valor = calcular()  
6 print(valor)
```

42

Funciones lambda ⚡

¿Qué es una lambda?

Función anónima de una sola línea, útil para operaciones simples

```
1 # Función normal
2 def cuadrado(x):
3     return x ** 2
4
5 # Equivalente como lambda
6 cuadrado = lambda x: x ** 2
7
8 print(cuadrado(5))
9 print(cuadrado(9))
```

➔ Salida

```
25
81
```

i Sintaxis: lambda parámetros: expresión

Caso de uso común

Las lambdas se usan frecuentemente como clave de ordenamiento

```
1 estudiantes = [  
2     ("Ana", 88),  
3     ("Luis", 95),  
4     ("María", 72)  
5 ]  
6  
7 # Ordenar por nota (segundo elemento)  
8 ordenados = sorted(estudiantes, key=lambda e: e[1])  
9  
10 for nombre, nota in ordenados:  
11     print(f"{nombre}: {nota}")
```

➡ Salida

María: 72 Ana: 88 Luis: 95

Recomendaciones

- ✓ Cada función debe hacer **una sola cosa** bien
- ✓ Usa nombres descriptivos: `calcular_area()` no `ca()`
- ✓ Mantén las funciones cortas (menos de 20 líneas)
- ✓ Documenta tus funciones con docstrings
- ✓ Prefiere `return` en lugar de modificar variables globales
- ✓ Prueba cada función de forma independiente

¿Qué es un docstring?

Cadena de texto que describe qué hace una función

```
1 def calcular_area(base, altura):
2     """
3     Calcula el área de un triángulo.
4
5     Args:
6         base (float): La base del triángulo.
7         altura (float): La altura del triángulo.
8
9     Returns:
10        float: El área calculada.
11    """
12    return (base * altura) / 2
13
14 help(calcular_area)
```

● Accede al docstring con `help(funcion)` o `funcion.__doc__`

Proyecto: Calculadora modular

```
1 def sumar(a, b):  
2     return a + b  
3  
4 def restar(a, b):  
5     return a - b  
6  
7 def multiplicar(a, b):  
8     return a * b  
9  
10 def dividir(a, b):  
11     if b == 0:  
12         raise ValueError("No se puede dividir por cero")  
13     return a / b
```

i Cada operación es una función independiente y reutilizable

Proyecto: Calculadora (continuación)

```
1 def calculadora():
2     operaciones = {
3         "+": sumar, "-": restar,
4         "*": multiplicar, "/": dividir
5     }
6     a = float(input("Primer número: "))
7     op = input("Operación (+, -, *, /): ")
8     b = float(input("Segundo número: "))
9
10    if op in operaciones:
11        resultado = operaciones[op](a, b)
12        print(f"Resultado: {resultado}")
13    else:
14        print("Operación no válida")
15
16 calculadora()
```

➔ Ejemplo de salida

Primer número: 10 Operación: * Segundo número: 5
Resultado: 50.0

¿Qué aprendimos hoy?

- ✓ Qué es una función y cómo definirla con `def`
- ✓ Parámetros, argumentos y valores por defecto
- ✓ Retorno de valores con `return`
- ✓ Alcance de variables (local vs global)
- ✓ Funciones lambda para expresiones simples
- ✓ Documentar funciones con docstrings
- ✓ Proyecto: Calculadora modular



¡Divide y conquista con funciones!



¡Preguntas!

Gracias por su atención